Gantry - Sprint 1

a) One safe move, step by step

1. Load the workspace grid and blocked-cell coordinates. Treat (0, 0) as the start position.

2. Read the operator’s target (x, y). If the target is outside the grid, print "Error: target out of bounds."

3. Check the target cell. If it is blocked, print "Error: target blocked."

4. Search for a path from (0, 0) to the target using A* search. Score each cell as f = g + h, where g is the number of steps from the start and h is the Manhattan distance to the target (|dx| + |dy|). Always expand the open cell with the lowest f; break ties by lower h, then by neighbor order: right, up, left, down. If a shorter path to an open cell is found, update its g and parent. Only move one cell horizontally or vertically; never enter a blocked or out-of-bounds cell.

5. If no path exists, print "Error: no route found." Otherwise, reconstruct the path from start to target.

6. Confirm homing is complete. If the system is not homed, it refuses the move and prints "Error: not homed."

7. Print "Route: " followed by every coordinate in order. If the target equals the start, print "Route: (0, 0)."

b) Pieces of the program

Ownership pending

Piece (class/file)	Its job	Receives	Hands on	Proposed owner
workspace.py / WorkspaceMap	Store grid dimensions and blocked cells; check whether a cell is valid.	Grid size, blocked-cell list	Valid-cell checks / grid to route finder	Lance
input_validator.py / InputValidator	Read target; reject unhomed, blocked, or out-of-bounds requests.	Target, homed flag, map	Validated target or error	Jay
pathfinder.py / Pathfinder	Run A* (Manhattan heuristic, fixed tie-break order); reconstruct safe route.	Map, start, valid target	Route or no-route result	Tex
controller.py / Controller	Coordinate validation and route search; manage homed state.	Target, map, homed flag	Route/error to output	Ricardo

main.py / CLI + tests	Set up example maps, run scenarios, print exact results and verify expected output.	Test cases and controller results	Printed routes/errors and test results	Imprezz

c) Five test cases

Coordinate convention: x increases right, y increases up. Neighbor search order: right, up, left, down. Start = (0, 0) in all cases.

Input	**Expected output**
1. 3 × 3 grid; blocked = {}; homed = yes; target = (2, 0).	Route: (0, 0) → (1, 0) → (2, 0).
2. 3 × 3 grid; blocked = {(1, 0)}; homed = yes; target = (2, 0).	Route: (0, 0) → (0, 1) → (1, 1) → (2, 1) → (2, 0).
3. 3 × 3 grid; blocked = {(1, 0), (0, 1)}; homed = yes; target = (2, 2).	Error: no route found.
4. 3 × 3 grid; blocked = {}; homed = yes; target = (3, 1).	Error: target out of bounds.
5. 3 × 3 grid; blocked = {}; homed = no; target = (1, 1).	Error: not homed.

d) What each of us needs to learn first

All developers: representing a grid with blocked cells, A* search with a priority queue and a Manhattan-distance heuristic, reconstructing a path, and agreeing on coordinate conventions and exact output messages. The map/validation owners should focus on boundary checks; the pathfinding owner on A* (cost/heuristic scoring and tie-breaking); the controller owner on homing state and error handling; the test owner on repeatable input/output tests.

Sprint 1 scope: laptop program only. Stop-button behavior, browser app, Fusion 360 model, and physical motor control are later steps, as directed by the professor.